

Object-Oriented Library Management System

A C++ library management system built specifically to demonstrate four classic design patterns working together in one real system.

Why Design Patterns, Not Just CRUD

A typical library system is a CRUD exercise: add/view/update/delete book records. This version is deliberately architected around four design patterns instead, so that the project doubles as concrete, defensible material for the extremely common interview question: *"Tell me about a design pattern you've used."*

Pattern 1: Factory (Book Creation)

`BookFactory::createBook(type, id, title, author)` centralizes creation of `FictionBook`, `NonFictionBook`, and `MagazineBook` objects. Calling code never references a concrete class directly — it just passes a type string. Each category has its own loan period (Fiction: 14 days, Non-Fiction: 21 days, Magazine: 7 days), so the factory also encodes real business rules per type.

Pattern 2: Singleton (Library Instance)

`Library::getInstance()` uses a Meyer's Singleton (a function-local static), guaranteeing exactly one `Library` instance exists for the whole program, is lazily constructed on first use, and is thread-safe by the C++11 standard's static initialization guarantee — without needing manual locking code.

Pattern 3: Observer (Book Reservation Waitlist)

Each `Book` is a `Subject` that maintains a waitlist of `Observer` pointers (Members who reserved it while checked out). When the book is returned, `setAvailable(true)` automatically calls `notifyObservers()`, which fires `onBookAvailable()` on every waiting member in order — verified below with two members waiting simultaneously.

Pattern 4: Strategy (Fine Calculation Policy)

Three interchangeable fine policies implement a common `FineStrategy` interface: `Standard` (flat rate/day), `Student` (50% discount), and `Grace-Period` (first N late days free). The `Library` swaps policies at runtime via `setFineStrategy(unique_ptr<FineStrategy>)` with zero changes to the borrowing/returning code.

Verified Program Output (actual run)

```
[3] Alice borrows "1984"...
    Issued "1984" to Alice (loan period: 14 days)

[4] Bob tries to borrow "1984" (unavailable, so he reserves it)...
    "1984" is currently checked out.
    Bob added to waitlist for "1984" (position 1)

[5] Carol also tries, and reserves it too (waitlist grows)...
    "1984" is currently checked out.
    Carol added to waitlist for "1984" (position 2)

[6] Alice returns "1984" 5 days late (Standard Fine Policy in effect)...
    "1984" returned (5 day(s) late). Fine [Standard Fine Policy]: Rs.25.00
    [Notification -> Bob]: The book "1984" (ID: B001) you reserved is now available!
    [Notification -> Carol]: The book "1984" (ID: B001) you reserved is now available!

[7] Switching to Student Discounted Fine Policy for Bob...
```

```
[Library] Fine policy switched to: Student Discounted Fine Policy
"Sapiens" returned (4 day(s) late). Fine [Student Discounted Fine Policy]: Rs.10.00
```

```
[8] Switching to Grace-Period Fine Policy (first 2 late days free)...
"National Geographic - July" returned (3 day(s) late). Fine [Grace-Period Fine Policy]: Rs.5.00
```

Both waiting members (Bob and Carol) were correctly notified in order the moment the book became available — confirming the Observer pattern fires exactly as intended. The same return flow produced three different fines (Rs.25, Rs.10, Rs.5) purely from swapping the active Strategy, with no changes to `returnBook()` itself.

Class Diagram (textual)

```
Observer (interface)          FineStrategy (interface)
|-- Member                    |-- StandardFineStrategy
                               |-- StudentFineStrategy
Book (abstract, Subject)      |-- GracePeriodFineStrategy
|-- FictionBook
|-- NonFictionBook            BookFactory --> creates --> Book subclasses
|-- MagazineBook

Library (Singleton)
- catalog: vector<unique_ptr<Book>>
- members: vector<unique_ptr<Member>>
- fineStrategy: unique_ptr<FineStrategy>
+ issueBook() / returnBook() / reserveBook()
```

Tech Stack

C++17, smart pointers (`unique_ptr`) for ownership and automatic memory management, abstract base classes for all four pattern interfaces, compiled with `g++ -O2`.

Possible Extensions

Persist catalog/members to a real database (JDBC/SQLite), add a Decorator pattern for book add-ons (e.g. audiobook version), and a Command pattern to support undo on issue/return operations.